

BOOBUZZ architecture -- DRAFT -- R3 in progress, final pass after robot-cx-13

This is the Phase 1.1 D1 architecture draft. It describes the implementation boundary and the R3 shape being applied in robot-code; it is not a replacement for protokol.md (owned by ftc-main) or for the historical Turkish mimari.md. The final pass is intentionally deferred until robot-cx-13 finishes and its cross-review is accepted.

The code and comments are English at the observed heads. This draft uses English class/package identifiers even when an older journey note still contains a Turkish or pre-R3 name.

1. Where the code lives

Repository and branch map

Repository	Branch	Local path	Observed
boobuzz-docs	stable	/home/shared/projects/boobuzz/docs	D1 A2 c
boobuzz-robotcode (local robot-code)	dev-phase-1.1	/home/shared/projects/boobuzz/robot-code	d4226eb
boobuzz-recocknize (local re-cock-nize)	dev-phase-1.1	/home/shared/projects/boobuzz/re-cock-nize	8af8281
boobuzz-ballautoistic (local ball-auto-istic)	project repository	/home/shared/projects/boobuzz/ball-auto-istic	not used

GitHub names were changed to the names in the first column; local folder names were deliberately left unchanged. The older documentation shorthand dev-phase-1 is the Phase 1 baseline. Phase 1.1 work is on dev-phase-1.1; the old cplx_engine_1 engine string remains accepted as a one-phase alias.

robot-code tree

```
robot-code/
|--- settings.gradle
|--- FtcRobotController/                # upstream FTC SDK module
|--- TeamCode/
|   |--- build.gradle                  # Android app; depends on :core
|   |--- core/                        # Gradle project :core
|       |--- build.gradle              # Java 17, SDK guard
|       `--- src/main/java/boobuzz/core/
|           |--- contract/             # DTOs and request records
|           |--- hal/                 # IHal, Mechanism, RobotConstants
|           |--- subsystem/
|               |--- pedro/            # Pedro 3.0 bridge
|               `--- stub/             # timing-only mechanisms
|           |--- logic/
|               |--- direct/           # DirectEngine and DirectMap
|               `--- cplx1/            # CplxEngine1 and logic modules
|           |--- controller/
|               |--- teleop/           # TeleopController and TeleopMap
|               |--- auto/             # AutoController/Builder/Sequence/Step/Runner
|               `--- opmodes/         # six routine data classes
|           |--- RobotLoop.java
|           `--- RobotFactory.java
|   `--- src/main/java/org/firstinspires/ftc/teamcode/
|       |--- hal/                     # Hardware and RealHal
|       `--- opmode/                  # TeleOp and autonomous shells
|--- sim/
```

```

|--- build.gradle                                # Gradle project :sim
`--- src/main/java/boobuzz/sim/
    |--- SimHal.java                            # TCP L1 adapter
    |--- SimMain.java                          # command-line runner
    |--- Json.java
    `--- *Exception.java

```

The tests mirror Java packages under TeamCode/core/src/test/java/boobuzz/core/ and simulator adapter tests are under sim/src/test/java/boobuzz/sim/.

re-cock-nize tree

```

re-cock-nize/
|--- sim/
|   |--- server.py                            # TCP protocol and lockstep
|   |--- mechanism.py                        # RobotConstants.java reader
|   |--- encoder.py                          # integer tick quantisation
|   |--- field.py                           # field dimensions and projection
|   |--- physics/
|       |--- backend.py                     # PhysicsBackend interface
|       |--- motor.py                      # shared motor/sensor model
|       |--- kinematic_backend.py          # Euler comparison backend
|       |--- pymunk_backend.py             # Pymunk rigid body
|       |--- pybullet_backend.py           # deterministic PyBullet body
|       |--- multi.py                      # shared world for multiple bodies
|   |--- assets/field_biobuzz.png          # optional viewer background
|   `--- viewer.py                         # pygame input/drawing
|--- tests/                               # backend, protocol, signal, viewer tests
|--- tools/fake_client.py                 # Java-free protocol client
`--- requirements.txt

```

The old sim/physics.py name is a compatibility alias in history; current physics implementations are in sim/physics/. carryover/ remains legacy and is not imported by the server.

Layer to package to class table

Layer/boundary	Package or module path	Key classes
Contract DTOs	boobuzz.core.contract	RobotState
HAL/configuration	boobuzz.core.hal	IHal, Mech
Subsystem interfaces	boobuzz.core.subsystem	ISubsystem
Pedro implementation	boobuzz.core.subsystem.pedro	PedroDrive
Mechanism stubs	boobuzz.core.subsystem.stub	StubShooter
Logic engines	boobuzz.core.logic and logic/direct, logic/cplx1	IRobotEng
Controllers	boobuzz.core.controller and controller/teleop, controller/auto, controller/opmodes	IController,
Tick/orchestration	boobuzz.core	RobotLoop
Real HAL and FTC shell	org.firstinspires.ftc.teamcode.hal and .opmode	Hardware,
Java simulator seam	boobuzz.sim in Gradle :sim	SimHal, Sim
Python physics seam	sim.*	SimServer,

RequestBatch/RequestStream and the removed Drive/Intent are the R3 contract at d4226eb. The header remains DRAFT because the requested final pass and cross-review are still a documentation gate, not because these classes are uncommitted.

2. Robot architecture

Runtime layers

The source tree has five core packages, but the runtime has three directional layers:

1. HAL (L1) owns clocks, hardware/simulator I/O, and raw sensor/action translation.
2. Logic (L2) includes the engine and the narrow subsystem implementations. It turns raw state plus controller messages into mechanism-level RobotAction values.
3. Controller (L3) owns policy: gamepad mapping, autonomous sequencing, and request creation. It returns DTOs and never reaches into a subsystem.

contract is a DTO-only seam between those layers. The root RobotLoop and RobotFactory are composition/orchestration; FTC SDK code is confined to TeamCode/src.

Ownership and forbidden knowledge

Component	Owns
IHal / RealHal / SimHal	now(), read(), write(), GamepadState source, hardware/socket details
Subsystem interfaces and implementations	One mechanism's observation, command state, and action contribution; Pedro
IRobotEngine and cplx/direct logic	Sensor-to-world projection, request arbitration, jobs, statuses, subsystem calls
IController implementations	Mapping and sequencing from Feedback to RequestBatch
contract records	Immutable message shapes and validation
RobotLoop / RobotFactory	Fixed call order and dependency construction

The dependency test enforces the same direction: contract imports no core package; hal imports contract; subsystem imports contract and Mechanism or RobotConstants (not IHal); logic imports contract and subsystem; controller imports contract; only root composition wires all parts.

One RobotLoop tick

The current R3 transition has this fixed single-threaded order:

1. RobotState state = hal.read().
2. WorldSnapshot snapshot = engine.sense(state). sense lets each subsystem observe the same sample.
3. Feedback feedback = new Feedback(snapshot, engine.drainStatuses(), state.t()). Statuses were produced by the preceding tick's act; this is deliberately one tick delayed.
4. RequestBatch batch = controller.decide(feedback).
5. engine.act(batch). The engine applies stream levels, cancellations, and edge-triggered requests to subsystem interfaces.
6. RobotAction action = engine.action(). Subsystems update in fixed drive, shooter, intake order (and turret after it is added).
7. hal.write(action).

hal.now() is the HAL clock; no wall-clock call orders a tick in core. RobotLoop.setEngine(IRobotEngine) is the explicit composition hook. A SWITCH_ENGINE(index) request is consumed by the loop before the current engine sees the batch, the old engine receives cancel-all, and the selected engine starts on the next tick over the same Subsystems object.

3. Simulator capabilities

Shared model and selectable backends

re-cock-nize is physics and rendering only; it contains no robot logic. The server selects one PhysicsBackend:

- kinematic: comparison baseline. It applies the shared motor model, derives chassis twist by least-squares mecanum inverse, Euler-integrates pose, and clamps the footprint to the 144-inch field.

- pymunk (default): deterministic planar rigid body in a pymunk.Space, four static walls, friction, traction forces at wheel positions, contacts, angled-wall rotation, and wall sliding.
- pybullet: deterministic planar body on a ground plane with four walls, force-at-wheel traction, fixed solver settings, and optional native GUI. It uses DIRECT in headless mode.

sim/physics/motor.py is shared by all three. It models each wheel separately: power is clamped to $[-1, 1]$, a calibrated free-speed target is multiplied by wheel efficiency, and output speed follows a first-order lag with $MOTOR_TAU_S = 0.1$ s. $BATTERY_V$ is carried in the mechanism and emitted as the state voltage; the current implementation does not model voltage sag, current draw, or a back-EMF circuit. Kinematic wheel-rim/chassis conversion is derived from each wheel's position and roller angle, not from handwritten front/rear signs. Encoder integration uses motor wheel output speed, then QuantizedEncoder emits integer ticks. Pinpoint and IMU values are noisy, seeded samples; truth remains separate.

The pygame Viewer is optional. It polls keyboard values into the protocol gamepad block, draws the 144-inch field, a centered 141-inch background image, axes, alliance edges, robot trail, truth pose, encoders, events, and a panel. --headless never imports pygame and runs without real-time pacing. Repeating a reset seed and exactly the same step sequence is deterministic; the seeded noise stream is part of the state sequence.

Coordinate conventions

The field is a 144 x 144 inch top view with its origin at a corner. Field +x is right, field +y is up, and heading h is counter-clockwise from +x. Robot-frame vx is forward, vy is left, and ω is counter-clockwise. The viewer's screen y axis is inverted only at the drawing boundary; the wire and Java/Python physics retain the field convention.

Calibration and origin

The Python reader consumes the same TeamCode/core/src/main/java/boobuzz/core/hal/RobotConstants.java file as Java. The comments identify the calibration source as last season's archive/ftc/de-cock/TeamCode/pedroPathing/Constan

Value	Current value	Origin/n
ROBOT_WIDTH, ROBOT_LENGTH	18.0 in, 18.0 in	current c
ROBOT_MASS_KG	12.0 kg	compile-
WHEEL_DIAMETER	4.0 in	current t
BATTERY_V	12.0 V	fixed bu
MOTOR_TAU_S	0.1 s	first-ord
STRAFE_EFF	0.7346	last-seas
zero-power forward decel	36.17 in/s ²	last-seas
zero-power lateral decel	85.98 in/s ²	last-seas
wheel roller angles	+45/-45 degrees	RobotC
wheel positions	+/-6.5 forward, +/-5.5 left	motor d
wheel ticks/rev	537.7	motor d
wheel free RPM	351.55735379568756	derived i
efficiency FL/FR/BL/BR	1.0 each	placehol
Pinpoint	(161.0, 0.0) mm; FORWARD/REVERSED; goBILDA_4_BAR_POD	last-seas
stub timing	spin-up 0.5 s; feed 0.2 s	Java-onl

What is and is not simulated

Simulated: motor lag and per-wheel speed, roller-angle mecanum kinematics, strafe efficiency, directional zero-power deceleration, integer encoder ticks, noisy Pinpoint/IMU, the planar chassis and wall contacts (Pymunk/PyBullet), optional gamepad input, event logging, and deterministic replay.

Not simulated: shooter, intake, turret, balls, goals, field elements, vision, battery sag/current/thermal behavior, servo motion, or a hardware Pinpoint device. Java StubShooter and StubIntake only time commands and

emit events; their state is not a force or a ball trajectory. `truth` is available to the viewer and `:sim` tests only and is never copied into `RobotState` or `logic`.

4. Logic engines

`cplx_engine_N` convention

Each complex engine is a sibling under `boobuzz.core.logic` (`cplx1`, future `cplx2`, and so on). An engine owns its own logic modules; engines do not share implementation code. Shared contracts and subsystem interfaces are the only intended reuse. During this transition `RobotFactory` accepts both `cplx1` and the historical `cplx_engine_1` string; canonical name is `cplx1`.

`cplx1` contents and status

The R3 target tree is:

```
logic/
|--- IRobotEngine.java
|--- direct/
|   |--- DirectEngine.java
|   `--- DirectMap.java
`--- cplx1/
    |--- CplxEngine1.java
    |--- MotionLogic.java
    |--- TurretLogic.java
    `--- ShooterLogic.java
```

The R3.8 commit `d4226eb` contains this complete tree. `CplxEngine1` owns `MotionLogic`, `TurretLogic`, and `ShooterLogic`; `DirectEngine` remains a separate wiring-only engine selected by `RobotFactory`, and `DirectMap` is its single request switch. The DRAFT marker remains until the requested final cross-review, but these classes are no longer merely planned files.

Pedro bridging is in `subsystem/pedro`, not in the engine package:

- `PedroDrive` implements `IDrive`, feeds `HalLocalizer` from `Pinpoint`, runs the Pedro Follower, and writes wheel powers through `HalDrivetrain`.
- `HalDrivetrain` implements Pedro's drivetrain seam and converts `DrivePowers` to named motor powers, with finite/clamped output protection.
- `HalLocalizer` implements Pedro Localizer; it applies a software pose offset and computes velocity from successive HAL samples.
- `PathRegistry` holds named test-line from (72,72,0) to (120,72,0) and test-turn hold target at (120,72,pi/2).
- `PedroConstants` constructs the Pedro follower with the shared mechanism.

The behaviours exposed by the R3 contract are:

Behaviour name	Current R3 representation
Manual	<code>RequestStream.manual(vx, vy, omega)</code> level for the current tick
Velocity	Same stream carries robot-frame velocity levels; no separate <code>Drive.Velocity</code> record
GoTo	<code>RequestType.GOTO</code> with x/y/heading parameters or a <code>PathRequest</code> target
FollowPath	<code>RequestType.PATH</code> carrying a named <code>PathRequest</code> or ordered segments
Hold	No active drive request means <code>IDrive.stop()</code> ; a path may also set <code>holdEnd</code>

`MotionLogic` owns stream/request arbitration: manual stream wins and rejects an active drive request; a tick with neither manual stream nor drive request stops the drive. `TurretLogic` aims at the configured goal every tick (or scans when pose is unavailable) and exposes a lock state. `ShooterLogic` implements IDLE ->

SPINNING -> FEEDING x count -> DONE, coordinating turret hold and shooter readiness. The final behavioural review is still a documentation gate.

Subsystem pattern and extension recipe

ISubsystem is deliberately small:

```
interface ISubsystem {
    void observe(RobotState state);
    void update(RobotAction.Builder out);
}
```

IDrive, IShooter, IIntake, and ITurret extend it. `Subsystems` is a fixed-order record (`IDrive drive`, `IShooter shooter`, `IIntake intake`, `ITurret turret`) with `observe(state)` and `update()`: drive, shooter, intake, then turret each write to the same action builder. Stubs are timing-only and individually unit-tested.

To add a subsystem:

1. Add its narrow I... extends ISubsystem interface under `boobuzz.core.subsystem`.
2. Add a testable implementation under `subsystem/stub` (and a real implementation when hardware is ready).
3. Add the field and fixed-order observe/update call to `Subsystems`, then construct it in `RobotFactory`.
4. Add the engine mapping in `DirectMap` (or the owning `cplx1` logic module) and a unit test; do not make the controller call the subsystem directly.

To add a request type, add the enum value, add one `DirectMap` switch case and its completion rule, then add controller/tests. `DirectMap` is intentionally the one small, editable type-dispatch file.

5. Inter-module communication (the critical seam)

Java contract records and methods

The following signatures are the source-level shapes at the R3 transition.

IHal extends IGamepadSource:

```
interface IHal extends IGamepadSource {
    long now(); // milliseconds, HAL/simulation clock
    RobotState read(); // one raw sample
    void write(RobotAction action); // one motor/servo/event output
}
```

IGamepadSource is `GamepadState get()`. `RealHal` fills it from FTC `gamepad1`; `SimHal` fills it from the server `gamepad` object.

```
record RobotState(long t,
    Map<String,Integer> enc,
    Map<String,Double> vel,
    double yaw,
    Pose pinpoint,
    double voltage) {}
record WorldSnapshot(long t, Pose pose, double yaw, double voltage) {}
record Feedback(WorldSnapshot world,
    List<RequestStatus> statuses,
    long t) {}
record Event(String name, long tMs, Map<String,Double> data) {}
record RobotAction(Map<String,Double> motors,
    Map<String,Double> servos,
```

```

        List<Event> events) {}
record GamepadState(double lx, double ly, double rx, double ry,
        boolean a, boolean b, boolean x, boolean y,
        boolean lb, boolean rb,
        double lt, double rt, Dpad dpad,
        boolean back, boolean start) {}
enum Dpad { NONE, UP, DOWN, LEFT, RIGHT }

```

RobotState.t is milliseconds; encoder values are integer ticks; vel is ticks/second; yaw and pinpoint.heading are radians; Pinpoint x/y are inches; voltage is volts. RobotState never contains simulator truth. Gamepad stick values are in [-1,1], triggers are in [0,1], and ly is negative when the physical stick is pushed forward (FTC/pygame convention). RobotAction values are named motor powers in [-1,1], named servo positions [0,1], and optional timestamped events. A missing motor/servo key is treated as zero by SimHal and RealHal.

IRobotEngine is bidirectional:

```

interface IRobotEngine {
    String name();
    WorldSnapshot sense(RobotState state);
    void act(RequestBatch batch);
    RobotAction action();
    List<RequestStatus> drainStatuses();
}
interface IController {
    RequestBatch decide(Feedback feedback);
}

```

The default action() and drainStatuses() implementations return zero/empty values for simple test engines. The compatibility overload sense(long now, RobotState state) is retained during migration.

Request, stream, batch, and status Request is an immutable edge-triggered message:

```

record Request(int id, RequestType type, double[] params, PathRequest path) {}
enum RequestType {
    SHOOT, INTAKE, GOTO, PATH, SPIN_UP, INTAKE_ON, INTAKE_OFF, TURN_TO,
    TURRET_AIM, RESET_POSE, SWITCH_ENGINE
}
record RequestStatus(int id, State state, double progress, String note) {
    enum State { ACCEPTED, ACTIVE, DONE, FAILED, REJECTED }
}

```

Request validates that only PATH carries a non-null PathRequest; params are a defensive copy. The first eight enum values were in the committed rename; TURRET_AIM, RESET_POSE, and SWITCH_ENGINE are present in the R3 working tree (turret aim is rejected by cplx1 because aiming is automatic). A drive request has an id and is answered with RequestStatus; status arrives in Feedback on the next tick.

RequestStream is the per-tick level message. It has no id and no completion answer:

```

record RequestStream(double vx, double vy, double omega, boolean manualDrive) {}
record RequestBatch(RequestStream stream,
        List<Request> requests,
        int[] cancels) {}

```

vx, vy, and omega are robot-frame forward, left, and counter-clockwise levels. manualDrive=false means no manual input this tick; an absent stream is the zero/idle stream. RequestBatch is the complete controller output for one tick: stream plus edge requests plus request IDs to cancel. A manual stream cancels/rejects active GOTO, PATH, or TURN_TO work with note overridden by manual drive; no stream and no drive request stops the drive.

The pre-R3 contract/Intent record and sealed contract/Drive variants are removed by R3. Their behaviour maps as follows:

Historical variant	R3 message
Drive.Manual(vx,vy,omega)	RequestStream with manualDrive=true
Drive.Velocity(...)	same stream (no second velocity DTO)
Drive.GoTo(target,constraints)	Request(GOTO, ...) / PathRequest.goTo
Drive.FollowPath(pathId)	Request(PATH, PathRequest.named(pathId))
Drive.Hold	no active request; drive stop/hold is the absence rule

PathRequest payload PathRequest is the immutable path value used by PATH and autonomous builders:

```
record PathRequest(String pathId,
    Pose target,
    Constraints constraints,
    List<Segment> segments,
    Heading heading,
    boolean holdEnd,
    Double velocityConstraint,
    Braking braking) {}
record Constraints(double maxPower, double maxVelocity) {}
enum HeadingMode { TANGENT, TANGENT_REVERSE, CONSTANT, LINEAR }
record Heading(HeadingMode mode, double start, double end) {}
record Braking(double strength, double startMultiplier) {}
sealed interface Segment { Pose end(); }
record Line(Pose end) implements Segment {}
record Curve(Pose end, List<Pose> controlPoints) implements Segment {}
```

A named request uses pathId; a direct target uses target; an inline path uses ordered Line/Curve segments. Headings and braking/velocity modifiers are carried in the request, in field inches/radians and Pedro constraints' native units.

Mechanism and compile-time configuration There is no current MechanismLoader and no mechanism.yaml. The old YAML schema is closed: configuration errors must fail at Java compile time, and removing a runtime parser leaves fewer moving parts. Mechanism.DEFAULT is the immutable runtime value built by RobotConstants.buildMechanism() and returned by RobotConstants.mechanism():

```
record Mechanism(List<String> motorNames,
    List<String> servoNames,
    Map<String, Motor> motors,
    Drivetrain drivetrain,
    Pinpoint pinpoint,
    Physics physics) {}
record Motor(String drives, double forward, double left, double freeRpm) {}
record Drivetrain(double wheelDiameter) {}
record Pinpoint(double xPodOffsetMm, double yPodOffsetMm,
    String xPodDirection, String yPodDirection, String podType) {}
record Physics(Map<String, Double> efficiency,
    double strafeEfficiency,
    double zeroPowerDecelForwardInchesPerSecondSquared,
    double zeroPowerDecelLateralInchesPerSecondSquared) {}
```

RobotConstants.java is the shared source. The Python sim/mechanism.py reader depends on these one-line forms (do not wrap a declaration or change the argument order):


```

public static final double NAME = value;
public static final String NAME = "value";
public static final Motor FL = new Motor("fl", "wheel", xForward, yLeft, rollerDeg, ticksPerRev, freeRpm);
public static final String[] SERVOS = {};
public static final Pinpoint PINPOINT = new Pinpoint(xOffsetMm, yOffsetMm, xDirection, yDirection, podT);

```

The actual motor declarations are one physical line in the Java file; the wrapped form above shows the grammar only. The Python regex reader requires ROBOT_WIDTH, ROBOT_LENGTH, WHEEL_DIAMETER, BATTERY_V, MOTOR_TAU_S, STRAFE_EFF, ZERO_POWER_DECEL_FORWARD_IN_S2, and ZERO_POWER_DECEL_LATERAL_IN_S2; it also reads the optional per-motor EFFICIENCY_<NAME> scalars (default 1.0). ROBOT_MASS_KG is matched but not stored in the Python Mechanism yet. String declarations are matched for ANGLE_UNIT and DRIVETRAIN_TYPE, with the latter selecting the drivetrain. Each motor has exactly seven arguments: name, drives, xForward, yLeft, rollerDeg, ticksPerRev, and freeRpm; Python converts roller degrees to radians. The Java MOTORS array defines Java's protocol motor order; Python currently preserves the order of the motor declarations rather than parsing that array. SERVOS defines servo names. PINPOINT has the Java line format above and is consumed by Java's RobotConstants; the current Python reader does not parse Pinpoint offsets. The remaining Java-only constants (alliance, goal, shooter, stub timing, and TeleOp reset values) are not part of the Python reader contract. Java validates the server ready name lists before a run continues.

Subsystems and events Subsystems.observe(state) calls each subsystem once; Subsystems.update() calls each in fixed order into one RobotAction.Builder. RealHal ignores events for hardware, while SimHal serializes them. Current stub event names are shooter.feed.start, shooter.feed.end, intake.on, and intake.off. The R3 turret stub adds turret.locked and turret.scan. Event payloads are not interpreted by physics.

Simulator JSON protocol

The wire transport is TCP on 127.0.0.1:5555, UTF-8, one JSON object per line. Python is the server and Java :sim is the client/clock owner. Protocol version is 1.

reset to ready Java sends:

```
{"type":"reset","seed":0,"pose":{"x":0.0,"y":0.0,"h":0.0}}
```

seed is an integer RNG seed. Pose x/y are field inches and h is field heading in radians. Python resets the selected backend, clears event/state logs, and returns:

```
{
  "type":"ready",
  "motors":["fl","fr","bl","br"],
  "servos":[],
  "proto":1,
  "state":{"
    "t_ms":0,
    "enc":{"fl":0,"fr":0,"bl":0,"br":0},
    "vel":{"fl":0.0,"fr":0.0,"bl":0.0,"br":0.0},
    "imu":{"yaw":0.0},
    "pinpoint":{"x":0.0,"y":0.0,"h":0.0},
    "voltage":12.0,
    "gamepad":{"lx":0.0,"ly":0.0,"rx":0.0,"ry":0.0,
      "a":false,"b":false,"x":false,"y":false,
      "lb":false,"rb":false,"lt":0.0,"rt":0.0,"dpad":"none"},
    "truth":{"x":0.0,"y":0.0,"h":0.0}
  }
}
```

ready.state is the initial state at t_ms=0; the first Java RobotLoop.tick() can call read() before physics time advances. dt_ms=0 is invalid for a normal step, but reset itself has no dt_ms.

step and state For every tick Java sends exactly one step:

```
{
  "type": "step",
  "dt_ms": 20,
  "motors": {"fl": 0.5, "fr": -0.3, "bl": 0.5, "br": -0.3},
  "servos": {},
  "events": [
    {"name": "shooter.feed.start", "t_ms": 1240, "data": {}}
  ]
}
```

dt_ms is a positive integer number of milliseconds and is the simulation time advanced by exactly this one step. Motor values are powers in [-1,1]; missing motor keys are zero. Servo values are positions in [0,1] and are transported for the HAL seam but have no physics model. events is optional and must be an array; each event has a non-empty string name, numeric finite t_ms (legacy key t is accepted), and an object data. The server records events in its state/event logs and viewer; it does not derive physics from them.

Python replies with a state object:

```
{
  "type": "state",
  "t_ms": 1240,
  "enc": {"fl": 1203, "fr": -870, "bl": 1199, "br": -865},
  "vel": {"fl": 2400.0, "fr": -1700.0, "bl": 2400.0, "br": -1700.0},
  "imu": {"yaw": 0.12},
  "pinpoint": {"x": 3.1, "y": 0.4, "h": 0.12},
  "voltage": 12.0,
  "gamepad": {"lx": 0.0, "ly": -0.8, "rx": 0.0, "ry": 0.0,
    "a": false, "b": false, "x": false, "y": false,
    "lb": false, "rb": false, "lt": 0.0, "rt": 0.0, "dpad": "none"},
  "truth": {"x": 3.0, "y": 0.5, "h": 0.12}
}
```

enc is integer motor ticks; vel is ticks/second; x/y lengths are inches; all headings are radians; voltage is volts. Gamepad sticks are [-1,1], triggers are [0,1], buttons are booleans, and dpad is one of none/up/down/left/right. The current viewer/server emits the 13 keys shown above; the newer Java `GamepadState.back` and `.start` fields are optional at this seam and `SimHal` treats missing keys as false. truth is deliberately not a `RobotState` field: only `SimHal.truth()` and the viewer may read it. The server does not advance physics until step arrives, and `SimHal.write()` blocks waiting for the matching state.

bye Java sends `{"type": "bye"}` when the client closes. The server closes that connection and waits for another client; there is no reply. Unknown message types, malformed JSON, invalid dt_ms, or malformed event arrays terminate the connection with a protocol error.

Allowed communication and shared state

Sender	Receiver	Allowed values
HAL	subsystem/engine	RobotState through read; the engine calls sense
HAL	controller	GamepadState through IGamepadSource only
Controller	engine	Feedback in; RequestBatch out
Engine	subsystem	narrow method calls (manual, follow, spinUp, feed, and so on)
Subsystem	engine	WorldSnapshot contribution and RobotAction.Builder contribution
Engine	HAL	completed RobotAction via RobotLoop
SimHal	Python server	JSON reset/step/bye
Python server	SimHal	JSON ready/state; truth is quarantined

Sender	Receiver	Allowed values
--------	----------	----------------

No controller may call a subsystem or write a motor. No subsystem may call a controller or inspect a request stream. No logic class may open the socket or read truth. The Python server never imports Java logic.

Shared mutable state is intentionally minimal: immutable static RobotConstants, the immutable Mechanism.DEFAULT graph, per-loop tick count, per-engine request jobs/status queue, per-subsystem private observation/action state, and Python backend pose/RNG/logs. There is no singleton engine, global request queue, or shared truth pose. An engine switch transfers the same subsystem object between engines; it does not introduce a global bus.

Tick sequence: simulator and real robot

The six design diagrams below use the same RobotLoop order. In a simulator, SimHal.read() returns the last state from the socket and SimHal.write() sends step and waits for the next state; on hardware, RealHal.read() samples Pinpoint/IMU/motors and RealHal.write() writes FTC devices. All upper-layer calls are identical.

The HAL substitution is the only simulator/robot branch; there is no isSim branch in core.

Six R3 request-flow diagrams

The following six diagrams are copied from phases/phase-1.1/request-flow.md; that file remains the diagram source of truth.

1. One tick

sequenceDiagram

```

    participant HAL as IHal
    participant SUB as Subsystems
    participant ENG as IRobotEngine (logic)
    participant CTL as IController
    HAL->>SUB: observe(RobotState)
    HAL->>ENG: sense(RobotState) -> WorldSnapshot
    ENG-->>CTL: Feedback(snapshot, drained RequestStatus[])
    CTL->>ENG: act(RequestBatch{stream, requests[], cancels[]})
    ENG->>SUB: IDrive / IShooter / IIntake / ITurret calls
    SUB->>HAL: update() -> RobotAction{motors, servos, events}

```

2. Two message kinds

flowchart LR

```

    GP[GamepadState] --> TM[TeleopMap + Buttons]
    AB[AutoBuilder sequence] --> SR[SequenceRunner]
    TM -->|every tick: vx,vy,omega| ST[RequestStream 'UDP'  
<br/>no id, no answer]
    TM -->|on edge: SHOOT, INTAKE_ON/OFF,<br/>RESET_POSE, SWITCH_ENGINE| RQ[Request 'TCP'  
<br/>id -> Req]
    SR -->|PATH, TURN_TO, SHOOT,<br/>SPIN_UP, INTAKE_*| RQ
    ST --> RB[RequestBatch]
    RQ --> RB
    RB --> ENG[IRobotEngine.act]

```

3. SHOOT in cplx1

sequenceDiagram

```

    participant C as Controller
    participant E as CplxEngine1

```

```

participant S as ShooterLogic
participant T as TurretLogic
participant SH as IShooter
participant TU as ITurret
C->>E: Request(id, SHOOT, count=3)
E->>S: requestShot(id, 3)
S->>SH: spinUp(rpmFor(distance))
loop every tick
    T->>TU: aimAt(goalX, goalY) (automatic, independent of SHOOT)
    S->>S: SPINNING: wait shooter.isReady && turretLogic.locked
end
S->>T: holdForShot(true)
loop count times
    S->>SH: feed() -> events shooter.feed.start / end
end
S->>T: holdForShot(false)
S-->>E: RequestStatus.done(id)
E-->>C: (next tick, in Feedback)

```

4. Drive arbitration

```

stateDiagram-v2
    [*] --> Stopped
    Stopped --> Manual: stream.manualDrive
    Stopped --> Following: Request GOTO/PATH/TURN_TO
    Following --> Manual: stream.manualDrive (request REJECTED "overridden")
    Following --> Stopped: drive.pathDone -> DONE
    Manual --> Stopped: !stream.manualDrive
    Manual --> Following: Request GOTO/PATH/TURN_TO

```

5. SHOOT in direct

```

flowchart LR
    RQ[Request SHOOT n] --> DE[DirectEngine wiring]
    DE --> DM[DirectMap case SHOOT]
    DM --> SH[IShooter.spinUp -> feed xn]
    SH -->|isFeeding false n times| DE
    DE -->|RequestStatus DONE| C[Controller]

```

6. Engine switch

```

sequenceDiagram
    participant C as TeleopController
    participant L as RobotLoop
    participant E1 as CplxEngine1
    participant E2 as DirectEngine
    C->>L: Request SWITCH_ENGINE(1) (start held 1 s)
    L->>E1: act(cancel-all batch)
    L->>L: engine = E2
    Note over L,E2: next tick sense/act go to E2, same Subsystems

```

6. Build and run

Gradle modules and SDK guard

robot-code/settings.gradle includes :FtcRobotController, :TeamCode, :core, and :sim; project(':core').projectDir = file('TeamCode/core'). :core and :sim target Java 17. :core is a Java library with the pure-Java Pedro 3.0 core artifact; :sim depends on :core and never on :TeamCode. Android :TeamCode owns RealHal, Hardware, and OpModes.

gradle/sdk-guard.gradle runs before compileJava in :core and :sim. It fails the build if source/test files in either pure-Java module mention com.qualcomm, org.firstinspires, or android. This is a build error, not a review convention.

Use Java 21 for the Gradle wrapper (the modules target Java 17):

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
./gradlew :core:test :sim:test
./gradlew :TeamCode:assembleDebug
```

Python simulator tests and server

```
cd /home/shared/projects/boobuzz/re-cock-nize
python -m venv .venv
.venv/bin/pip install -r requirements.txt
PYTHON="$PWD/.venv/bin/python" ./run_tests.sh
.venv/bin/python -m sim.server \
    --mechanism /home/shared/projects/boobuzz/robot-code/TeamCode/core/src/main/java/boobuzz/core/hal/Robo
    --physics pymunk --headless --port 5556
```

Use --physics kinematic for comparison, --physics pybullet --headless for deterministic Bullet, or --physics pybullet --gui for the native Bullet window. Omitting --headless selects the pygame viewer for Pymunk/kinematic. Do not install npm tooling for this documentation pipeline.

Java simulator and end-to-end

Start the Python server first, then run the Java client:

```
cd /home/shared/projects/boobuzz/robot-code
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
./gradlew :sim:run --args="--port 5556 --path test-line --x 72 --y 72 --h 0 --dt 20 --steps 1000 --seed
```

SimMain also accepts --engine direct|cplx1 (and temporary cplx_engine_1 alias), --drive vx,vy,omega, and --auto . Physics is selected on the Python server, not in Java. Registry names are BlueDoggy6Piece, RedDoggy6Piece, BlueMissionary9Piece, RedMissionary9Piece, BlueMissionary9PieceLever, and RedMissionary9PieceLever.

Appendix A -- Per-class reference

Class/file	Short reference
IHal	HAL clock/read/write and gamepad seam
RealHal	FTC HardwareMap, Pinpoint, motors, servos, voltage, gamepad1 adapter
SimHal	TCP client; reset handshake; one step/state exchange; truth quarantine
RobotState	Raw sensor DTO
WorldSnapshot	Logic-facing pose/yaw/voltage view
RobotAction / Event	Named outputs and timestamped subsystem events
Mechanism / RobotConstants	Immutable runtime config and compile-time source
ISubsystem / Subsystems	Observe/update lifecycle and fixed ordering

Class/file	Short reference
PedroDrive	Pedro follower, path requests, manual powers
HalDrivetrain	Pedro DrivePowers to named, finite motor powers
HalLocalizer	Pinpoint sample bridge and software pose offset
PathRegistry / PedroConstants	Named test paths and follower construction
StubShooter / StubIntake / StubTurret	Timing/state stubs with events
IRobotEngine	Sense/act/action/status engine seam
DirectEngine	Request jobs, stream arbitration, cancellation, direct wiring
DirectMap	Editable request-to-subsystem switch used by DirectEngine
CplxEngine1	Complex engine composing motion, turret, shooter, and intake logic
MotionLogic / TurretLogic / ShooterLogic	Drive arbitration, automatic aim, and shot state machine
IController	Feedback to RequestBatch policy seam
TeleopController	Gamepad source and field-oriented drive mapping
Buttons / TeleopMap	Edge/toggle helper and editable control map
AutoController / SequenceRunner	Advance AutoSequence from statuses/time
AutoBuilder / AutoSequence / AutoStep	Fluent autonomous data and cursor
AutoRegistry and six routine classes	Names and builders for last season's autos
RobotLoop / RobotFactory	Fixed tick and engine/subsystem composition
SimServer	Python TCP protocol, lockstep, event/state logs
Mechanism reader	Regexes RobotConstants.java; converts degrees to radians
PhysicsBackend	Reset/step/state backend seam
MotorModel / QuantizedEncoder	Shared lag, kinematics, noise, tick quantisation
KinematicBackend	Euler/clamp comparison
PymunkBackend / PyBulletBackend	Rigid-body/contact backends
Viewer	Optional pygame controls and field rendering

Appendix B -- Open decisions and closed decisions

- R3 contract freeze: wait for robot-cx-13 implementation, tests, and Codex cross-review before removing the DRAFT marker.
- Pinpoint offsets: historical documents contain 161/0 and -84/-168 alternatives. RobotConstants.PINPOINT is currently 161.0/0.0; hardware re-measurement must choose final offsets.
- Nested Android Studio module sync: :core physically lives under TeamCode/core; IDE indexing/sync behaviour still needs clean Android Studio verification.
- R3 engine switch ownership: SWITCH_ENGINE belongs to RobotLoop, not an engine; exact index validation and cancel-all status policy remain to be reviewed.
- RobotConstants record duplication: RobotConstants.Motor/Pinpoint and Mechanism.Motor/Pinpoint currently duplicate data shapes. Fold them into one representation in the next cleanup.
- Mass source: ROBOT_MASS_KG exists in Java, while both rigid backends still use literal 12.0; make the backends read the constant through Mechanism in cleanup.
- YAML decision -- closed: mechanism.yaml, SnakeYAML, and MechanismLoader were removed. Compile-time failures and fewer moving parts are the rationale.
- No physical turret/shooter yet: stub-to-hardware interfaces stay narrow until real mechanisms exist.

Appendix C -- Contradictions to resolve (not silently merged)

These are source discrepancies observed while preparing the draft; this document does not choose a winner:

1. The v1 portion of phases/phase-1.1/design-spec.md names Hal, Drive, Intent, and the old controller/autos tree; its appended R3 revision names IHal, RequestBatch, cplx1, and controller/opmodes.
2. robot-cx-13-r3-naming-logic-requests.md is written against pre-R3 2573880, while the observed branch has since landed R3.1-R3.8 through d4226eb. The task file's snapshot therefore calls several now-present classes "pending"; the code and the task are not at the same revision.

3. Older README, architecture, and Phase 1 task notes still use `mechanism.yaml`, `MechanismLoader`, or `--mechanism mechanism.yaml`; current robot/simulator code reads `RobotConstants.java`.
4. Older architecture text says the simulator is kinematic and PyBullet is not simulated; current re-cock-nize has Pymunk and PyBullet backends plus the kinematic comparison backend.
5. `protokol.md` historical examples use `GamepadSource`, `Hal`, and a possible 12.6 V state, while current names are `IGamepadSource`, `IHal`, and `RobotConstants.BATTERY_V = 12.0`.
6. The old `Drive.Manual` wording and current R3 `RequestStream` wording describe the same robot-frame values but different DTO names; R3 removal is still under review.
7. The first request-flow Mermaid diagram models `HAL->Subsystem observe` as a separate edge, while current `IRobotEngine.sense` invokes `Subsystems.observe` internally; the diagram is retained verbatim as the design source.
8. `GamepadState` now has `back/start` fields, but the current Python viewer omits those JSON keys and `SimHal` defaults missing keys to false.
9. `mimari.md` is an intentionally stale Turkish historical inventory and references legacy Python modules; it is not a second current architecture source.